

Design Uber

⚡ Real-time Updates

☰ Multi-step Processes

By Evan King · Published Jul 18, 2024 · 🏠 hard · Asked at:    



Try This Problem Yourself

Practice with guided hints and real-time feedback



Start Practice



Watch Video Walkthrough

Watch the author walk through the problem step-by-step



Watch Now

Understanding the Problem

What is Uber?

Uber is a ride-sharing platform that connects passengers with drivers who offer transportation services in personal vehicles. It allows users to book rides on-demand from their smartphones, matching them with a nearby driver who will take them from their location to their desired destination.

Functional Requirements



1. Start your interview by defining the functional and non-functional requirements. For user facing applications like this one, functional requirements are the "Users should be able to..." statements whereas non-functional defines the system qualities via "The system should..." statements.
2. Prioritize the top 3 functional requirements. Everything else shows your product thinking, but clearly note it as "below the line" so the interviewer knows you won't be including them in your design. Check in to see if your interviewer wants to move anything above the line or move anything down. Choosing just the top 3 is important to ensuring you stay focused and can execute in the limited time window.

Core Requirements

1. Riders should be able to input a start location and a destination and get a fare estimate.
2. Riders should be able to request a ride based on the estimated fare.
3. Upon request, riders should be matched with a driver who is nearby and available.
4. Drivers should be able to accept/decline a request and navigate to pickup/drop-off.

Below the line (out of scope)

1. Riders should be able to rate their ride and driver post-trip.
2. Drivers should be able to rate passengers.
3. Riders should be able to schedule rides in advance.

- Riders should be able to request different categories of rides (e.g., X, XL, Comfort).

Non-Functional Requirements

Core Requirements

- The system should prioritize low latency matching (< 1 minutes to match or failure)
- The system should ensure strong consistency in ride matching to prevent any driver from being assigned multiple rides simultaneously
- The system should be able to handle high throughput, especially during peak hours or special events (100k requests from same location)

Below the line (out of scope)

- The system should ensure the security and privacy of user and driver data, complying with regulations like GDPR.
- The system should be resilient to failures, with redundancy and failover mechanisms in place.
- The system should have robust monitoring, logging, and alerting to quickly identify and resolve issues.
- The system should facilitate easy updates and maintenance without significant downtime (CI/CD pipelines).

Functional Requirements

- Riders request estimated fare
- Riders expect fare to get matched with a driver
- Driver can accept/deny and navigate to pickup/dropoff

----- Out of Scope -----

- Rating drivers/riders
- Scheduling rides in advance
- Support different ride types (X, XL, etc)

Non-functional Requirements

- Low latency ride matching (< 1 minute)
- Strong consistency for matching
- Handle surges (100k from same loc. @ same time)

----- Out of Scope -----

- GDPR
- Backups
- CI/CD
- Fault tolerance
- Monitoring



Uber Requirements



Adding features that are out of scope is a "nice to have". It shows product thinking and gives your interviewer a chance to help you reprioritize based on what they want to see in the interview. That said, it's very much a nice to have. If additional features are not coming to you quickly, don't waste your time and move on.

The Set Up

Planning the Approach

Before you move on to designing the system, it's important to start by taking a moment to plan your strategy. Fortunately, for these common users facing product-style questions, the plan should be straightforward: build your design up sequentially, going one by one through your functional requirements. This will help you stay focused and

ensure you don't get lost in the weeds as you go. Once you've satisfied the functional requirements, you'll rely on your non-functional requirements to guide you through the deep dives.

Defining the Core Entities

I like to begin with a broad overview of the primary entities. At this stage, it is not necessary to know every specific column or detail. We will focus on the intricacies, such as columns and fields, later when we have a clearer grasp. Initially, establishing these key entities will guide our thought process and lay a solid foundation as we progress towards defining the API.

To satisfy our key functional requirements, we'll need the following entities:

1. **Rider:** This is any user who uses the platform to request rides. It includes personal information such as name and contact details, preferred payment methods for ride transactions, etc.
2. **Driver:** This is any users who are registered as drivers on the platform and provide transportation services. It has their personal details, vehicle information (make, model, year, etc.), and preferences, and availability status.
3. **Fare:** This entity represents an estimated fare for a ride. It includes the pickup and destination locations, the estimated fare, and the estimated time of arrival. This could also just be put on the ride object, but we'll keep it separate for now but there is no right or wrong answer here.
4. **Ride:** This entity represents an individual ride from the moment a rider requests an estimated fare all the way until its completion. It records all pertinent details of the ride, including the identities of the rider and the driver, vehicle details, state, the planned route, the actual fare charged at the end of the trip, and timestamps marking the pickup and drop-off.
5. **Location:** This entity stores the real-time location of drivers. It includes the latitude and longitude coordinates, as well as the timestamp of the last update. This entity is crucial for matching riders with nearby drivers and for tracking the progress of a ride.

In the actual interview, this can be as simple as a short list like this. Just make sure you talk through the entities with your interviewer to ensure you are on the same page.

Core Entities

- Driver
- Rider
- Fare
- Ride
- Location

Uber Core Entities

Now, let's proceed to design our system, tackling each functional requirement in sequence. This step-by-step approach will help us maintain focus and manage scope effectively, ensuring a cohesive build-up of the system's architecture.



As you move onto the design, your objective is simple: create a system that meets all functional and non-functional

requirements. To do this, I recommend you start by satisfying the functional requirements and then layer in the non-functional requirements afterward. This will help you stay focused and ensure you don't get lost in the weeds as you go.

API or System Interface

The API for retrieving a fare estimate is straightforward. We define a simple POST endpoint that takes in the user's current location and desired destination and returns a Fare object with the estimated fare and eta. We use POST here because we will be creating a new ride object in the database.

```
POST /fare -> Fare
Body: {
  pickupLocation,
  destination
}
```

Request Ride Endpoint: This endpoint is used by riders to confirm their ride request after reviewing the estimated fare. It initiates the ride matching process by signaling the backend to find a suitable driver, thus creating a new ride object.

```
POST /rides -> Ride
Body: {
  fareId
}
```



Note that at this point in the flow, we match them with a driver who is nearby and available. However, this is all happening in the backend, so we don't need to explicitly list out an endpoint for this.

Update Driver Location Endpoint: Before we can do any matching, we need to know where our drivers are. This endpoint is used by drivers to update their location in real-time. It is called periodically by the driver client to ensure that the driver's location is always up to date.

```
POST /drivers/location -> Success/Error
Body: {
  lat, long
}
```

- note the driverId is present in the session cookie or JWT and not in the body or path params



Always consider the security implications of your API. I regularly see candidates passing in data like `userId`, `timestamps`, or even `fareEstimate` in the body or query parameters. This is a red flag as it shows a lack of understanding of security best practices. Remember that you can't trust any data sent from the client as it can be easily manipulated. User data should always be passed in the session or JWT, while timestamps should be generated by the server. Data like `fareEstimate` should be retrieved from the database and never passed in by the client.

Accept Ride Request Endpoint: This endpoint allows drivers to accept a ride request. Upon acceptance, the system

...that request endpoint. The endpoint then either to accept a request, open acceptance, the system updates the ride status and provides the driver with the pickup location coordinates.

```
PATCH /rides/:rideId -> Ride
Body: {
  accept/deny
}
```

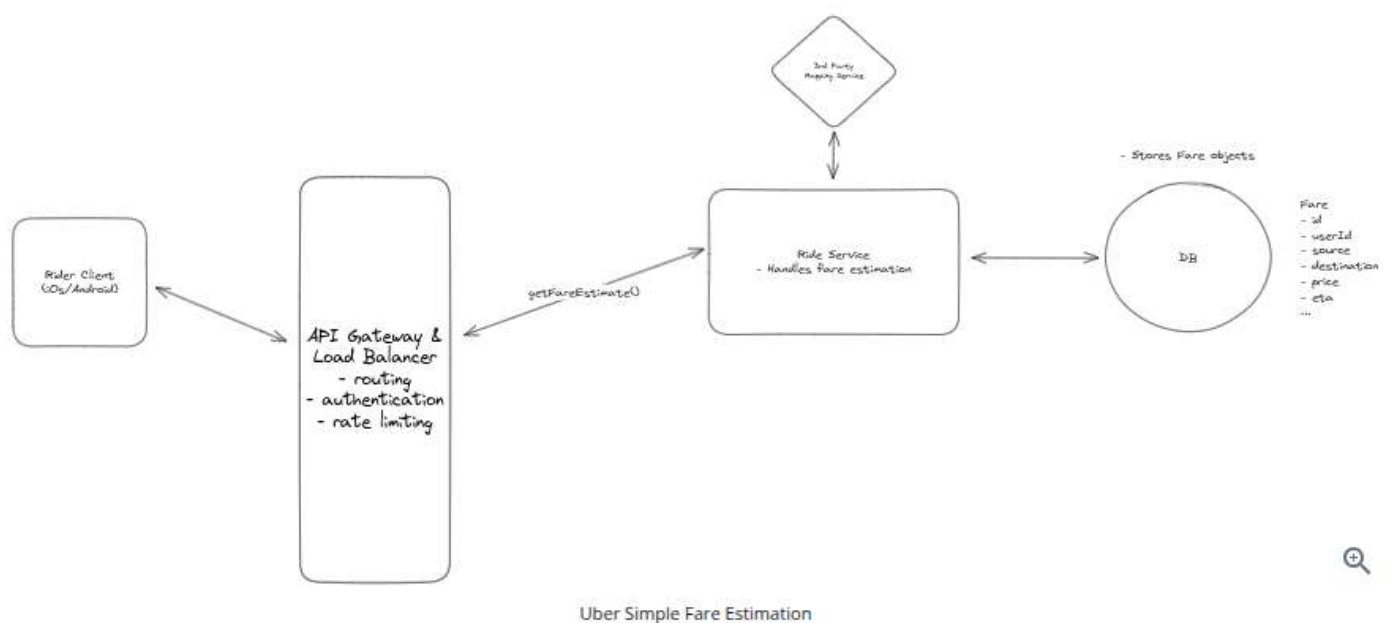
The `Ride` object is would contain information about the pickup location and destination so the client can display this information to the driver.

High-Level Design

1) Riders should be able to input a start location and a destination and get an estimated fare

The first thing that users will do when they open the app to request a ride is search for their desired destination. At this point, the client will make a request to our service to get an estimated price for the ride. The user will then have a chance to request a ride with this fare or do nothing.

Lets lay out the necessary components for communicating between the client and our microservices, adding our first service, "Ride Service" which will handle fare estimations



The core components necessary to fulfill fare estimation are:

- 1. Rider Client:** The primary touchpoint for users is the Rider Client, available on iOS and Android. This client interfaces with the system's backend services to provide a seamless user experience.
- 2. API Gateway:** Acting as the entry point for client requests, the API Gateway routes requests to the appropriate microservices. It also manages cross-cutting concerns such as authentication and rate limiting.
- 3. Ride Service:** This microservice is tasked with managing ride state, starting with calculating fare estimates. It interacts with third-party mapping APIs to determine the distance and travel time between locations and applies the company's pricing model to generate a fare estimate. For the sake of this interview, we abstract

this complexity away.

4. **Third Party Mapping API:** We use a third-party service (like Google Maps) to provide mapping and routing functionality. It is used by the Ride Service to calculate the distance and travel time between locations.
5. **Database:** The database is, so far, responsible for storing Fare entities. In this case, it creates a fare with information about the price, eta, etc.

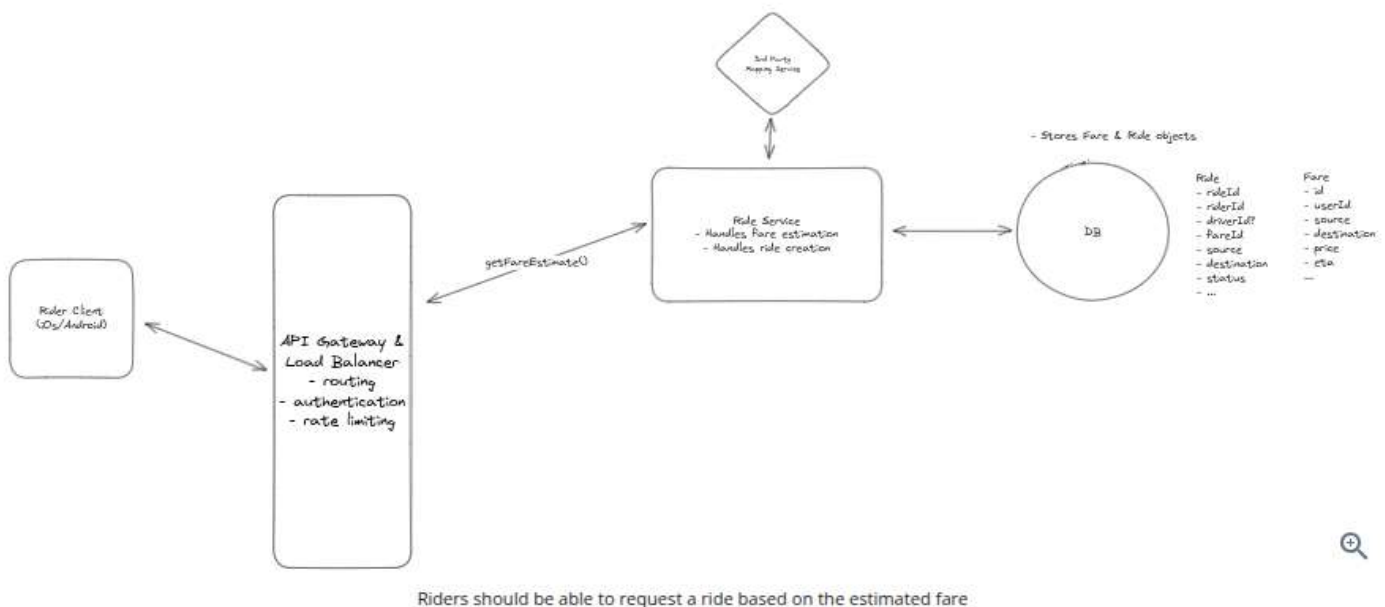
Let's walk through exactly how these component interact when a rider requests a fare estimate.

1. The rider enters their pickup location and desired destination into the client app, which sends a POST request to our backend system via `/fare`
2. The API gateway receives the request and handles any necessary authentication and rate limiting before forwarding the request to the Ride Service.
3. The Ride Service makes a request to the Third Party Mapping API to calculate the distance and travel time between the pickup and destination locations and then applies the company's pricing model to the distance and travel time to generate a fare estimate.
4. The Ride Service creates a new Fare entity in the Database with the details about the estimated fare.
5. The service then returns the Fare entity to the API Gateway, which forwards it to the Rider Client so they can make a decision about whether accept the fare and request a ride.

2) Riders should be able to request a ride based on the estimated fare

Once a user reviews the estimated fare and ETA, they can request a ride. By building upon our existing design, we can extend it to support ride requests pretty easily.

We don't need to add any new entities at all, we just need to add a Ride table to our Database.



Then, when a request comes in, this is how we handle it.

1. The user confirms their ride request in the client app, which sends a POST request to our backend system with the id of the Fare they are accepting.
2. The API gateway performs necessary authentication and rate limiting before forwarding the request to the Ride Service.
3. The Ride Service receives the request and creates a new entry in the Ride table, linking to the relevant Fare

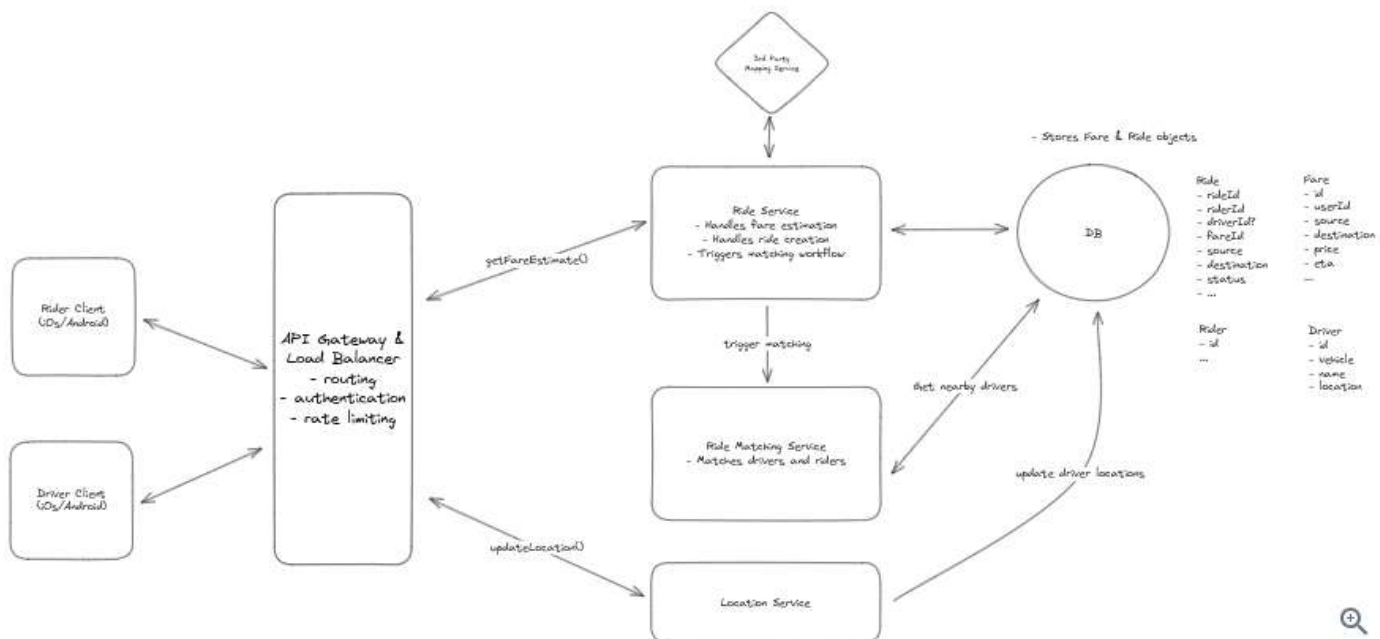
that was accepted, and initializing the Ride's status as `requested`.

4. Next, it triggers the matching flow so that we can assign a driver to the ride (see below)

3) Upon request, riders should be matched with a driver who is nearby and available

Now we need to introduce some new components in order to facilitate driver matching.

1. **Driver Client:** In addition to the Rider Client, we introduce the Driver Client, which is the interface for drivers to receive ride requests and provide location updates. The Driver Client communicates with the Location Service to send real-time location updates.
2. **Location Service:** Manages the real-time location data of drivers. It is responsible for receiving location updates from drivers, storing this information in the database, and providing the Ride Matching Service with the latest location data to facilitate accurate and efficient driver matching.
3. **Ride Matching Service:** Handles incoming ride requests and utilizes a sophisticated algorithm (abstracted away for the purpose of this interview) to match these requests with the best available drivers based on proximity, availability, driver rating, and other relevant factors.



Upon request, riders should be matched with a driver who is nearby and available

Let's walk through the sequence of events that occur when a user requests a ride and the system matches them with a nearby driver:

1. The user confirms their ride request in the client app, which sends a POST request to our backend system with the ID of the fare they are accepting.
2. The API gateway performs necessary authentication and rate limiting before forwarding the request to the Ride Matching Service.
3. We create a ride object as mentioned above, and then we trigger the matching workflow (we'll discuss ways to make this more robust later, keeping it simple for now).
4. Meanwhile, at all times, drivers are sending their current location to the location service, and we are updating our database with their latest location lat & long so we know where they are.
5. The matching workflow then uses these updated locations to query for the closest available drivers in an

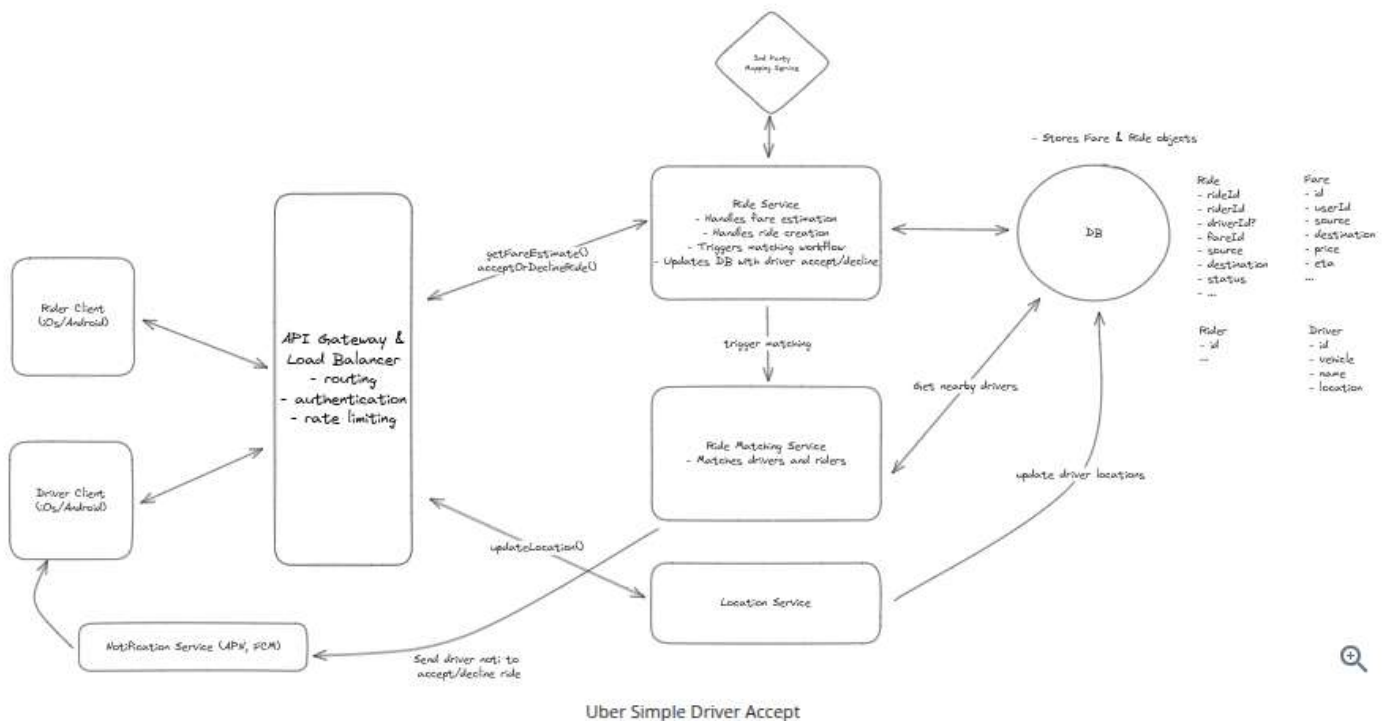
attempt to find an optimal match.

4) Drivers should be able to accept/decline a request and navigate to pickup/drop-off

Once a driver is matched with a rider, they can accept the ride request and navigate to the pickup location.

We only need to add one additional service to our existing design.

1. **Notification Service:** Responsible for dispatching real-time notifications to drivers when a new ride request is matched to them. It ensures that drivers are promptly informed so they can accept ride requests in a timely manner, thus maintaining a fluid user experience. Notifications are sent via APN (Apple Push Notification) and FCM (Firebase Cloud Messaging) for iOS and Android devices, respectively.



Let's walk through the sequence of events that occur when a driver accepts a ride request and completes the ride:

1. After the Ride Matching Service determines the ranked list of eligible drivers, it sends a notification to the top driver on the list via APN or FCM.
2. The driver receives a notification that a new ride request is available. They open the Driver Client app and accept the ride request, which sends a PATCH request to our backend system with the `rideID`. a) If they decline the ride instead, the system will send a notification to the next driver on the list.
3. The API gateway receives the requests and routes it to the Ride Service.
4. The Ride Service receives the request and updates the status of the ride to "accepted" and updates the assigned driver accordingly. It then returns the pickup location coordinates to the Driver Client.
5. With the coordinates in hand, the Driver uses on client GPS to navigate to the pickup location.

⚡ Pattern: Real-time Updates

Interviewer looking for a push notification to drivers? Our realtime updates pattern breakdown walks through the options from long-polling to SSE to Websockets.

Potential Deep Dives

With the core functional requirements met, it's time to dig into the non-functional requirements via deep dives. These are the main deep dives I like to cover for this question.



The degree to which a candidate should proactively lead the deep dives is a function of their seniority. For example, it is completely reasonable in a mid-level interview for the interviewer to drive the majority of the deep dives. However, in senior and staff+ interviews, the level of agency and ownership expected of the candidate increases. They should be able to proactively look around corners and identify potential issues with their design, proposing solutions to address them.

1) How do we handle frequent driver location updates and efficient proximity searches on location data?

Managing the high volume of location updates from drivers and performing efficient proximity searches to match them with nearby ride requests is a difficult task, and our current high-level design most definitely does not handle this well. There are two main problems with our current design that we need to solve:

1. **High Frequency of Writes:** Given we have around 10 million drivers, sending locations roughly every 5 seconds, that's about 2 million updates a second! Whether we choose something like DynamoDB or PostgreSQL (both great choices for the rest of the system), either one would either fall over under the write load, or need to be scaled up so much that it becomes prohibitively expensive for most companies.
2. **Query Efficiency:** Without any optimizations, to query a table based on lat/long we would need to perform a full table scan, calculating the distance between each driver's location and the rider's location. This would be extremely inefficient, especially with millions of drivers. Even with indexing on lat/long columns, traditional B-tree indexes are not well-suited for multi-dimensional data like geographical coordinates, leading to suboptimal query performance for proximity searches. This is essentially a non-starter.



For DynamoDB in particular, 2M writes a second of ~100 bytes would cost you about 100k a day. Learn more about DynamoDB and its limitations [here](#)

So, what can we do to address these issues?

Bad Solution: Direct Database Writes and Proximity Queries



Approach

The bad approach is what we briefly describe above, and what we have coming out of our high-level design. This approach involves directly writing each driver's location update to the database as it comes in, and performing proximity searches on this raw data. It's considered a bad approach because it doesn't scale well with the high volume of updates from millions of drivers, and it makes proximity searches inefficient and slow. This method would likely lead to system overload, high latency, and poor user experience, making it unsuitable for a large-

would likely lead to system overload, high latency, and poor user experience, making it unsuitable for a large-scale ride-sharing application like this.

Good Solution: Batch Processing and Specialized Geospatial Database ^

Approach

Instead of writing each driver location update directly to the database, updates are aggregated over a short interval and then batch-processed. This reduces the number of write operations. For proximity searches, a specialized geospatial database with appropriate indexing is used to efficiently query nearby drivers.

Batch processing is a common technique for reducing the number of write operations to a database. It involves collecting data over a predefined time interval and then writing it to the database in a single batch operation. This can significantly reduce the number of write transactions, which can help in improving write throughput and reducing lock contention.

For proximity searches, a specialized geospatial database with appropriate indexing is used to efficiently query nearby drivers. Geospatial databases use specialized data structures, such as quad-trees, to index driver locations, allowing for faster execution of proximity searches. Quad-trees are particularly well-suited for two-dimensional spatial data like geographic coordinates, as they partition the space into quadrants recursively, which can significantly speed up the process of finding all drivers within a certain area around a ride request.

If we were to go with PostgreSQL, it actually support a plugin called PostGIS that allows us to use geospatial data types and functions without needing an additional database.

Challenges

The interval between batch writes introduces a delay, which means the location data in the database may not reflect the drivers' most current positions. This can lead to suboptimal driver matches.

Great Solution: Real-Time In-Memory Geospatial Data Store ^

Approach

We can address all the limitation of the previous solutions by using an in-memory data store like Redis, which supports geospatial data types and commands. This allows us to handle real-time driver location updates and proximity searches with high throughput and low latency while minimizing storage costs with automatic data expiration.

Redis is an in-memory data store that supports geospatial data types and commands. It uses geohashing to encode latitude and longitude coordinates into a single string key, which is then indexed using a sorted set. This allows for efficient storage and querying of geospatial data.

Redis provides geospatial commands like GEOADD for adding location data and GEOSEARCH for querying nearby locations within a given radius or bounding box. These commands are highly optimized for geospatial data and can be used to efficiently handle real-time driver location updates and proximity searches. The GEOSEARCH

command, introduced in Redis 6.2, replaces and enhances the functionality of the older `GEORADIUS` and `GEORADIUSBYMEMBER` commands, offering more flexibility and performance improvements.

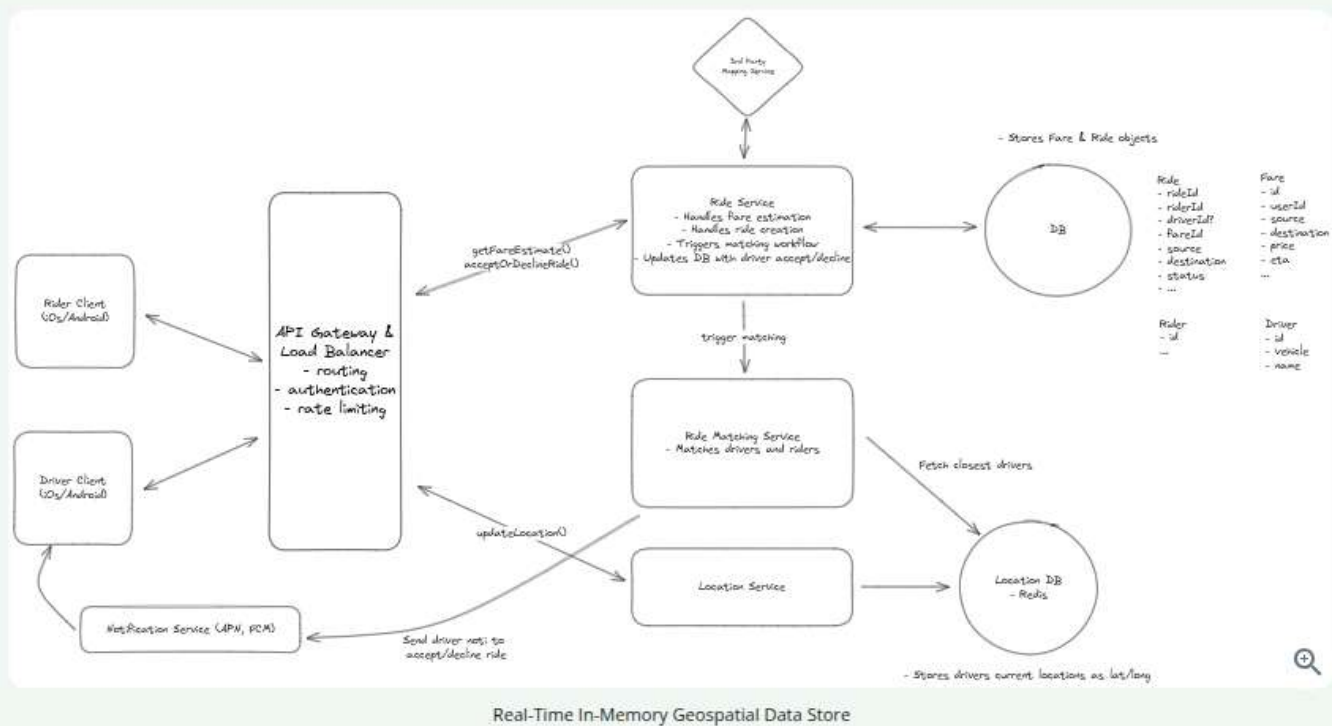
We no longer have a need for batch processing since Redis can handle the high volume of location updates in real-time. Additionally, Redis automatically expires data based on a specified time-to-live (TTL), which allows us to retain only the most recent location updates and avoid unnecessary storage costs.

Challenges

The main challenge with this approach would be durability. Since Redis is an in-memory data store, there's a risk of data loss in case of a system crash or power failure. However, this risk can be mitigated in a few ways:

1. Redis persistence: We could enable Redis persistence mechanisms like RDB (Redis Database) or AOF (Append-Only File) to periodically save the in-memory data to disk.
2. Redis Sentinel: We could use Redis Sentinel for high availability. Sentinel provides automatic failover if the master node goes down, ensuring that a replica is promoted to master.

Even if we experience data loss due to a system failure, the impact would be minimal. Since driver location updates come in every 5 seconds, we would only need that long to recover and rebuild the state of our system. This quick recovery time ensures that our system remains robust and reliable, even in the face of potential failures.



2) How can we manage system overload from frequent driver location updates while ensuring location accuracy?

High-frequency location updates from drivers can lead to system overload, straining server resources and network bandwidth. This overload risks slowing down the system, leading to delayed location updates and potentially impacting user experience. In most candidates original design, they have drivers ping a new location every 5 seconds or so. This follow up question is designed to see if they can intelligently reduce the number of pings while

maintaining accuracy.

Great Solution: Adaptive Location Update Intervals

Approach

We can address this issue by implementing adaptive location update intervals, which dynamically adjust the frequency of location updates based on contextual factors such as speed, direction of travel, proximity to pending ride requests, and driver status. This allows us to reduce the number of location updates while maintaining accuracy.

The driver's app uses ondevice sensors and algorithms to determine the optimal interval for sending location updates. For example, if the driver is stationary or moving slowly, updates can be less frequent. Conversely, if the driver is moving quickly or changing direction often, updates are sent more frequently.

Challenges

The main challenge with this approach is the complexity of designing effective algorithms to determine the optimal update frequency. This requires careful consideration and testing to ensure accuracy and reliability. But, if done well, it will significantly reduce the number of location updates and improve system efficiency.



Don't neglect the client when thinking about your design. Many candidates get in a habit of drawing a small client box and moving on. In many cases, we need client side logic to improve the efficiency and scalability of our system. As you saw, we can reduce the number of pings by using ondevice sensors and algorithms to determine the optimal interval for sending location updates. Similarly, in the case of a file upload service, the client is responsible for chunking and compression.

3) How do we prevent multiple ride requests from being sent to the same driver simultaneously?

We defined consistency in ride matching as a key non-functional requirement. This means that we only request one driver at a time for a given ride request AND that each driver only receives one ride request at a time. That driver would then have 10 seconds to accept or deny the request before we move on to the next driver if necessary. If you've solved Ticketmaster before, you know this problem well -- as it's almost exactly the same as ensuring that a ticket is only sold once while being reserved for a specific amount of time at checkout.

Bad Solution: Application-Level Locking with Manual Timeout Checks

Approach

The key intuition is that we need to lock drivers to prevent multiple ride requests from being sent to the same driver simultaneously. The question then just becomes, "how"? One approach is to use application-level locking, where each instance of the Ride Matching Service marks a ride request as "locked" when it is sent to a driver. It then starts a timer for the lock duration. If the driver does not accept the ride within this period, the instance manually releases the lock and makes the request available to other drivers.

Problems

This approach has several issues, here are the main ones:

- **Lack of Coordination:** With multiple instances of the Ride Matching Service running, there's no centralized coordination, leading to potential race conditions where two instances might attempt to lock the same ride request simultaneously.
- **Inconsistent Lock State:** If one instance sets a lock but fails before releasing it (due to a crash or network issue), other instances have no knowledge of this lock, which can leave the ride request in a locked state indefinitely.
- **Scalability Concerns:** As the system scales and the number of instances increases, the problem of coordinating locks across instances becomes more pronounced, leading to a higher likelihood of errors and inconsistencies.

Good Solution: Database Status Update with Timeout Handling ^

Approach

To solve the coordination problem, we can move the lock into the database. This allows us to use the database's built-in transactional capabilities to ensure that only one instance can lock a ride request at a time. When we send a request to a driver, we would update the status of that driver to "outstanding_request", or something similar. If the driver accepts the request, we update the status to "accepted" and if they deny it, we update the status to "available". We can then use a simple timeout mechanism within the Ride Service to ensure that the lock is released if the driver does not respond within the 10 second window.

Challenges

While we solved the coordination issue, we still run into issues with relying on a in-memory timeout to handle unlocking if the driver does not respond. If the Ride Service crashes or is restarted, the timeout will be lost and the lock will remain indefinitely. This is a common issue with in-memory timeouts and why they should be avoided when possible. One solution is to have a cron job that runs periodically to check for locks that have expired and release them. This works, but adds unnecessary complexity and introduces a delay in unlocking the ride request.

Great Solution: Distributed Lock with TTL ^

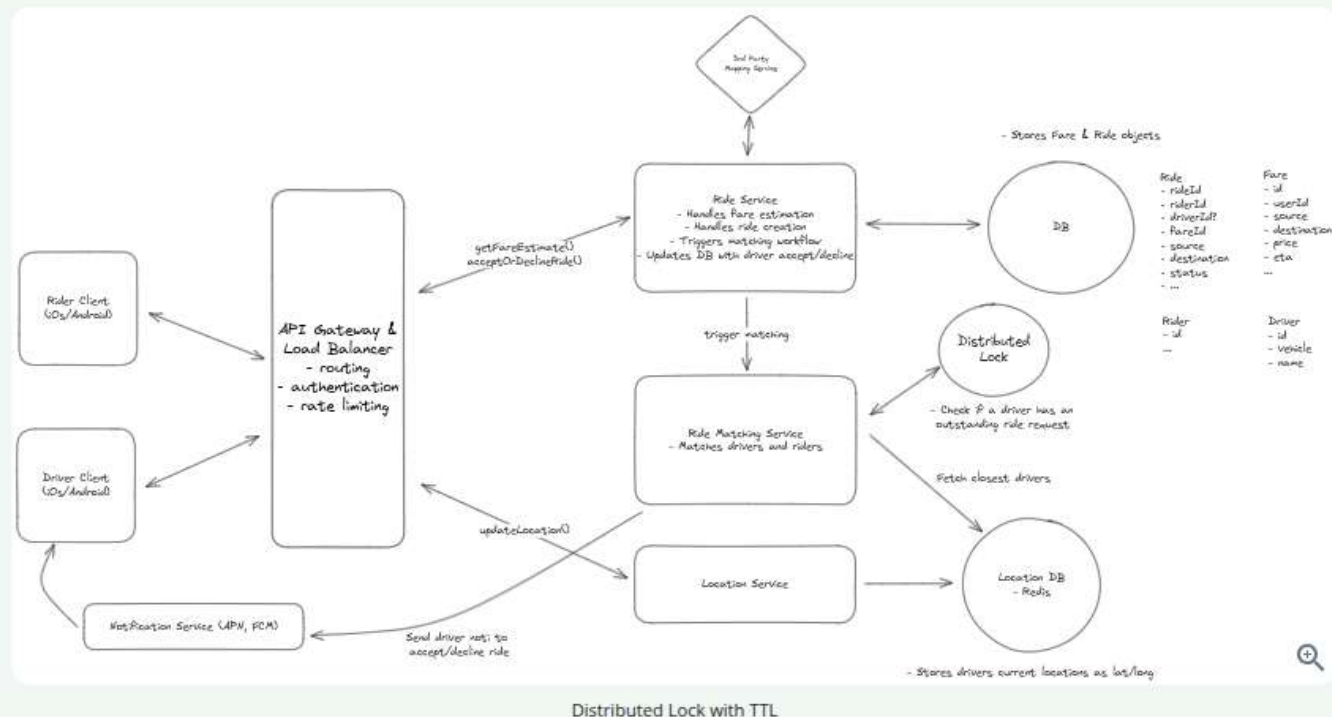
Approach

To solve the timeout issue, we can use a distributed lock implemented with an in-memory data store like Redis. When a ride request is sent to a driver, a lock is created with a unique identifier (e.g., `driverId`) and a TTL set to the acceptance window duration of 10 seconds. The Ride Matching Service attempts to acquire a lock on the `driverId` in Redis. If the lock is successfully acquired, it means no other service instance can send a ride request to the same driver until the lock expires or is released. If the driver accepts the ride within the TTL window, the

Ride Matching Service updates the ride status to "accepted" in the database, and the lock is released in Redis. If the driver does not accept the ride within the TTL window, the lock in Redis expires automatically. This expiration allows the Ride Matching Service to consider the driver for new ride requests again.

Challenges

The main challenge with this approach is the system's reliance on the availability and performance of the in-memory data store for locking. This requires robust monitoring and failover strategies to ensure that the system can recover quickly from failures and that locks are not lost or corrupted. Given locks are only held for 10 seconds, this is a reasonable tradeoff as the ephemerality of the data makes it easier to recover from failures.



4) How can we ensure no ride requests are dropped during peak demand periods?

During peak demand periods, the system may receive a high volume of ride requests, which can lead to dropped requests. This is particularly problematic during special events or holidays when demand is high and the system is under stress. We also need to protect against the case where an instance of the Ride Matching Service crashes or is restarted, leading to dropped rides.

Bad Solution: First-Come, First-Served with No Queue

Approach

The simplest approach is to process ride requests as they come in without any queuing system (as is currently done in our design). This is a first-come, first-served approach and what we currently have in our high-level design.

Challenges

The main issue with this approach is that it does not scale well during high-demand periods. As the number of

incoming requests increases, the system may become overwhelmed and start dropping requests, leading to a poor user experience. We can scale our ride matching service horizontally to handle more requests, but with sudden surges in demand, we may not be able to scale quickly enough to prevent dropped requests.

Additionally, if a Ride Matching Service goes down, any ride requests that were being processed by that instance would be lost. This could result in riders waiting indefinitely for a match that will never come, leading to a poor user experience and potential loss of customers. There's no built-in mechanism for request recovery or failover in this approach.

Great Solution: Queue with Dynamic Scaling

Approach

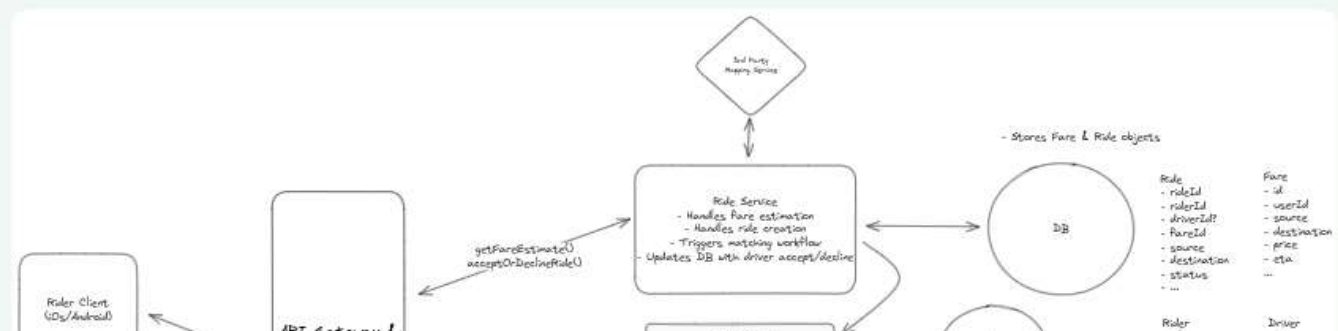
To address this issue, we can introduce a queueing system with dynamic scaling. When a ride request comes in, it is added to the queue. The Ride Matching Service then processes requests from the queue in a first-come, first-served manner. If the queue grows too large, the system scales horizontally by adding more instances of the Ride Matching Service to handle the increased load. This allows us to scale the system dynamically based on demand, ensuring that no requests are dropped. We can also partition the queues based on geographic regions to further improve efficiency.

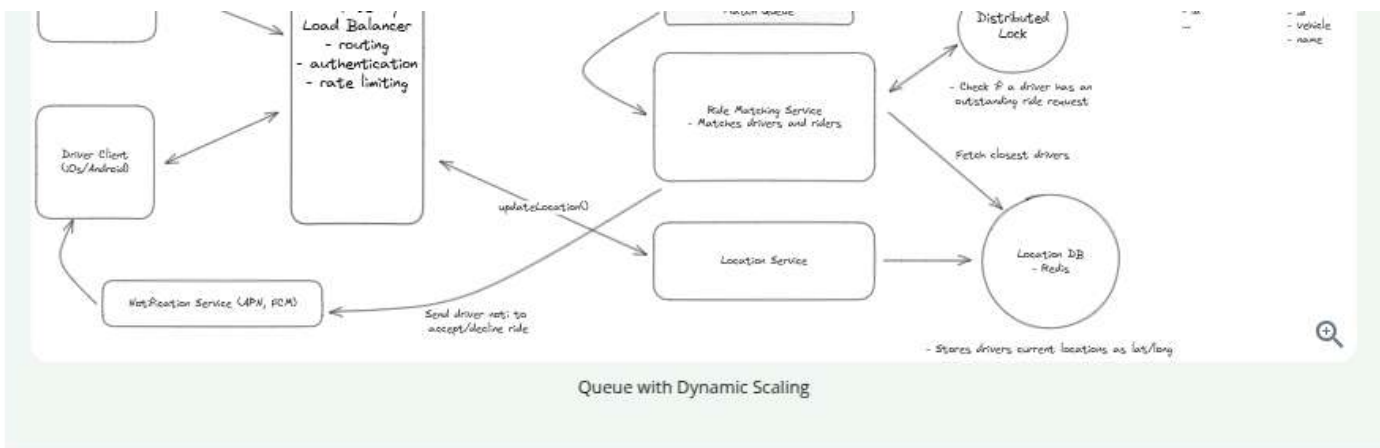
We could use a distributed message queue system like Kafka, which allows us to commit the offset of the message in the queue only after we have successfully found a match. This way, if the Ride Matching Service goes down, the match request would still be in the queue, and a new instance of the service would pick it up. This approach ensures that no ride requests are lost due to service failures and provides fault tolerance to our system.

Challenges

The main challenge with this approach is the complexity of managing a queueing system. We need to ensure that the queue is scalable, fault-tolerant, and highly available. We can address this by using a managed queueing service like Amazon SQS or Kafka, which provides these capabilities out of the box. This allows us to focus on the business logic of the system without worrying about the underlying infrastructure.

The other issue with this approach is that since it is a FIFO queue you could have requests that are stuck behind a request that is taking a long time to process. This is a common issue with FIFO queues and can be addressed by using a priority queue instead. This allows us to prioritize requests based on factors like driver proximity, driver rating, and other relevant factors. This ensures that the most important requests are processed first, leading to a better user experience.





5) What happens if a driver fails to respond in a timely manner?

Our system works great when the drivers either accept or deny the ride request, but what if they drop their phone in the passenger seat and take a break? How do we ensure that the ride request continues to be processed? Ideally we'd want the system to move on to the next driver if the current driver doesn't respond in a timely manner.

Pattern: Multi-step Processes

Human-in-the-loop processes are a big signal of when to use the Multi-Step Processes pattern. In fact, Uber is the original author of the open source project Cadence which gave birth to the leading durable execution framework Temporal, built specifically for use-cases like this one.

[Learn This Pattern](#)

Good Solution: Delay queue

Approach

We can implement a delay queue mechanism to automatically retry ride requests with the next available driver if the current driver doesn't respond within the timeout period. When a ride request is sent to a driver, we simultaneously schedule a delayed message in a queue (like Amazon SQS delay queue) that gets processed after the 10-second timeout period.

The delayed message contains the ride request details and the driver who was originally contacted. When the delayed message is processed, the system checks if the ride is still unassigned. If so, it automatically moves to the next driver in the ranked list and sends them the ride request, while scheduling another delayed message for the new driver.

This approach ensures that ride requests continue to be processed even if drivers don't respond, maintaining system resilience and improving user experience by reducing wait times.

Challenges

The main challenge with this approach is managing the complexity of multiple delayed messages and ensuring they are properly canceled when a driver accepts the ride. If a driver accepts the ride after the delayed message is already scheduled but before it's processed, we need to ensure the delayed message is handled gracefully and

doesn't cause the ride to be reassigned incorrectly.

Additionally, this approach requires careful coordination between the delay queue and the ride matching service to ensure consistency and avoid race conditions.

Great Solution: Durable Execution

Approach

A more robust solution is to use a durable execution framework like [Temporal](#) or AWS Step Functions. These systems provide built-in support for timeouts, retries, and state management in a way that survives service crashes and restarts.

With durable execution, the entire ride matching workflow is modeled as a durable workflow that can handle complex business logic, including driver timeouts, retries, and fallback mechanisms. The workflow maintains its state persistently, so even if the service crashes, the workflow can be resumed from where it left off.

For example, a Temporal workflow would:

1. Send the ride request to the first driver
2. Set a 10-second timeout
3. If the driver accepts, complete the workflow
4. If the driver declines or times out, automatically move to the next driver
5. Continue this process until a driver is found or all drivers are exhausted

The entire process is fault-tolerant and can handle service failures, network issues, and other disruptions without losing state or dropping ride requests.

Challenges

The main challenge with this approach is the additional complexity of introducing a workflow orchestration system. It requires teams to learn new concepts and tools, and adds another component to the system that needs to be monitored and maintained.

However, the benefits of guaranteed execution, built-in fault tolerance, and simplified business logic often outweigh these challenges, especially for mission-critical systems like ride-sharing platforms where dropped requests directly impact revenue and user experience.

6) How can you further scale the system to reduce latency and improve throughput?

Bad Solution: Vertical Scaling

Approach

The simplest approach to scaling is vertical scaling, where we increase the capacity of our existing servers by

adding more CPU, memory, or storage. This is a quick and easy way to increase capacity, but it has several limitations.

Challenges

This solution sucks for many reasons. First, it is expensive and requires downtime to upgrade the servers. Second, it is not infinitely scalable. At some point, we will hit the limits of vertical scaling and need to scale horizontally. Lastly, it is not fault tolerant. If the server fails, the system goes down. We can address this by adding redundancy, but that is a bandaid and not a real solution. In an interview, it's hardly worth discussing this options as it's not practical for a system of this scale.

Great Solution: Geo-Sharding with Read Replicas

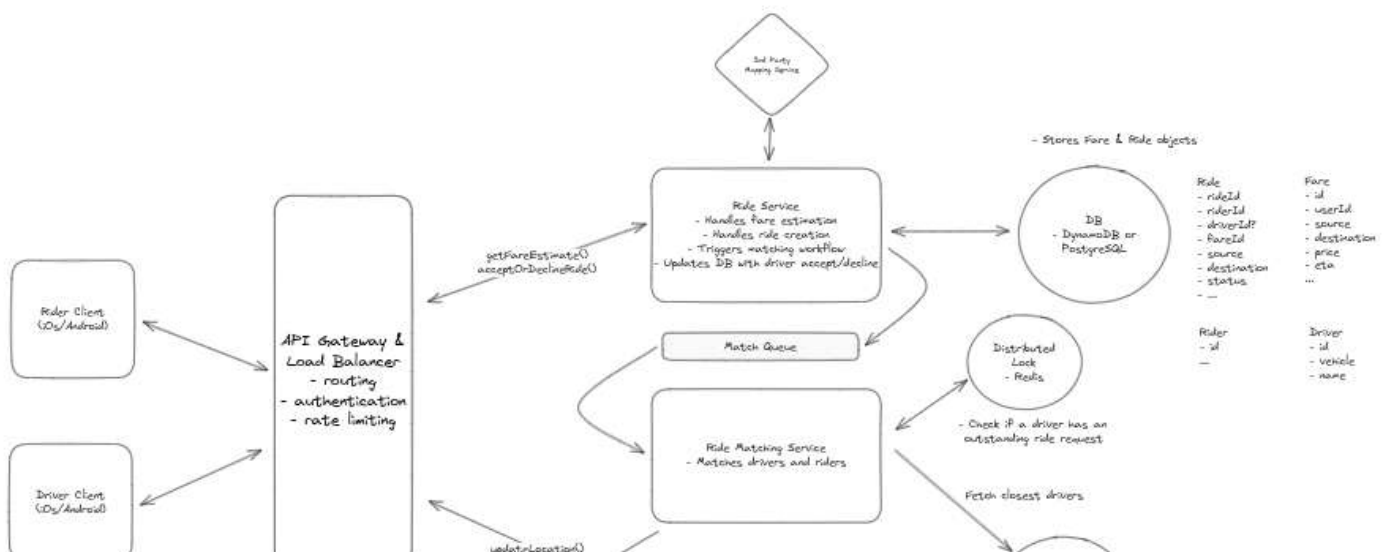
Approach

A better approach is to scale horizontally by adding more servers. We can do this by sharding our data geographically and using read replicas to improve read throughput. This allows us to scale the system to handle more requests while reducing latency and improving throughput. Importantly, this not only allows us to scale but it reduces latency by reducing the distance between the client and the server. This applies to everything from our services, message queue, to our databases -- all of which can be sharded geographically. The only time that we would need to scatter gather (i.e., query multiple shards) is when we are doing a proximity search on a boundary.

Challenges

The main challenge with this approach is the complexity of sharding and managing multiple servers. We need to ensure that data is distributed evenly across shards and that the system can handle failures and rebalancing. We can address this by using consistent hashing to distribute data across shards and by implementing a replication strategy to ensure that data is replicated across multiple servers. This allows us to scale the system horizontally while maintaining fault tolerance and high availability.

After applying the "Great" solutions, your updated whiteboard should look something like this:





What is Expected at Each Level?

Ok, that was a lot. You may be thinking, "how much of that is actually required from me in an interview?" Let's break it down.

Mid-level

Breadth vs. Depth: A mid-level candidate will be mostly focused on breadth (80% vs 20%). You should be able to craft a high-level design that meets the functional requirements you've defined, but many of the components will be abstractions with which you only have surface-level familiarity.

Probing the Basics: Your interviewer will spend some time probing the basics to confirm that you know what each component in your system does. For example, if you add an API Gateway, expect that they may ask you what it does and how it works (at a high level). In short, the interviewer is not taking anything for granted with respect to your knowledge.

Mixture of Driving and Taking the Backseat: You should drive the early stages of the interview in particular, but the interviewer doesn't expect that you are able to proactively recognize problems in your design with high precision. Because of this, it's reasonable that they will take over and drive the later stages of the interview while probing your design.

The Bar for Uber: For this question, an E4 candidate will have clearly defined the API endpoints and data model, landed on a high-level design that is functional and meets the requirements. They would have understood the need for some spatial index to speed up location searches, but may not have landed on a specific solution. They would have also implemented at least the "good solution" for the ride request locking problem.

Senior

Depth of Expertise: As a senior candidate, expectations shift towards more in-depth knowledge — about 60% breadth and 40% depth. This means you should be able to go into technical details in areas where you have hands-on experience. It's crucial that you demonstrate a deep understanding of key concepts and technologies relevant to the task at hand.

Advanced System Design: You should be familiar with advanced system design principles. For example, knowing how to use a search-optimized data store like [Elasticsearch](#) for event searching is essential. You're also expected to understand the use of a distributed cache or similar for locking drivers and to discuss detailed scaling strategies (it's ok if this took some probing/hints from the interviewer), including sharding and replication. Your ability to navigate these advanced topics with confidence and clarity is key.

Articulating Architectural Decisions: You should be able to clearly articulate the pros and cons of different architectural choices, especially how they impact scalability, performance, and maintainability. You justify your decisions and explain the trade-offs involved in your design choices.

Problem-Solving and Proactivity: You should demonstrate strong problem-solving skills and a proactive approach. This includes anticipating potential challenges in your designs and suggesting improvements. You need to be adept at identifying and addressing bottlenecks, optimizing performance, and ensuring system reliability.

The Bar for Uber: For this question, E5 candidates are expected to speed through the initial high level design so you can spend time discussing, in detail, at least 2 of the solutions to speed up location searches, the ride request locking problem, or the ride request queueing problem. You should also be able to discuss the pros and cons of different architectural choices, especially how they impact scalability, performance, and maintainability.

Staff+

Emphasis on Depth: As a staff+ candidate, the expectation is a deep dive into the nuances of system design — I'm looking for about 40% breadth and 60% depth in your understanding. This level is all about demonstrating that, while you may not have solved this particular problem before, you have solved enough problems in the real world to be able to confidently design a solution backed by your experience.

You should know which technologies to use, not just in theory but in practice, and be able to draw from your past experiences to explain how they'd be applied to solve specific problems effectively. The interviewer knows you know the small stuff (REST API, data normalization, etc) so you can breeze through that at a high level so you have time to get into what is interesting.

High Degree of Proactivity: At this level, an exceptional degree of proactivity is expected. You should be able to identify and solve issues independently, demonstrating a strong ability to recognize and address the core challenges in system design. This involves not just responding to problems as they arise but anticipating them and implementing preemptive solutions. Your interviewer should intervene only to focus, not to steer.

Practical Application of Technology: You should be well-versed in the practical application of various technologies. Your experience should guide the conversation, showing a clear understanding of how different tools and systems can be configured in real-world scenarios to meet specific requirements.

Complex Problem-Solving and Decision-Making: Your problem-solving skills should be top-notch. This means not only being able to tackle complex technical challenges but also making informed decisions that consider various factors such as scalability, performance, reliability, and maintenance.

Advanced System Design and Scalability: Your approach to system design should be advanced, focusing on scalability and reliability, especially under high load conditions. This includes a thorough understanding of distributed systems, load balancing, caching strategies, and other advanced concepts necessary for building robust, scalable systems.

The Bar for Uber: For a staff+ candidate, expectations are high regarding depth and quality of solutions, particularly for the complex scenarios discussed earlier. Great candidates are diving deep into at least 3+ key areas, showcasing not just proficiency but also innovative thinking and optimal solution-finding abilities. A crucial indicator of a staff+ candidate's caliber is the level of insight and knowledge they bring to the table. A good measure for this is if the interviewer comes away from the discussion having gained new understanding or perspectives.



Test Your Knowledge

Take a quick 15 question quiz to test what you've learned.

 [Start Quiz](#)

How would you rate the quality of this article?

1

2

3

4

5

6

7

8

9

10

PoorGreat

Add a comment...

B *I* `</>` **99**

Comment

☐ Anonymous

Posting as leetcode

Sort By
Popular **U** **UniversalCyanQuelea456** Top 1% • 1 year ago

Additionally, Redis automatically expires data based on a specified time-to-live (TTL), which allows us to retain only the most recent location updates and avoid unnecessary storage costs.

How would you suggest implementing this? I don't think Redis TTLs can be directly used for expiring individual drivers' location data. Geospatial indexes in Redis operate on sorted sets as the underlying structure. Individual members in a sorted set can't be given a TTL.

Possible ideas:

- As a form of expiration we could add a TTL to the sorted set itself and periodically switch to writing and reading to a new key with some time bucket. E.g. something like "driver-locations-2024-03-28-18-01". This can be as granular as necessary to avoid getting too much stale data, though does increase load somewhat as we may have to check the current and previous bucket and aggregate them.
- Have a separate K-V for each driver with its own TTL, which is updated simultaneously with the driver location. Then you can use the expired key space notification to trigger a deletion. This is supported in Redis Stack or using the RedisGears module.
- Forgo Redis TTL entirely and just use two sorted sets, one for the location and one for the driver last updated times or expiration time. Have some external process periodically read the update times (`ZRANGEBYSCORE`) and update both sets.

[Show More](#) 13 [Reply](#)**Evan King** Admin • 1 year ago

Nice, these are great options! And you're completely correct on the observations. Pros and cons to each of these, but I may opt for option 2 here.

 3 [Reply](#)**S****socialguy** Top 5% • 1 year ago

In option 2, I assume the key is the driver id, in which case, we will have to scan whole Redis to find a match. If the key is the location instead, assuming it's a 6-char geohash, it's not going to be unique to be a key.

Instead, we could have sorted sets by geohash, the score = timestamp (desc) and the member = driver id. Every 10s, the

location service/another cron would drop the members that are $> \text{now} - 10\text{s}$. Just to be sure, the ride matching service would check in the while loop that the driver's last location update was $< 10\text{s}$ ago before sending a notification.

👍 1 [Reply](#)

S **socialguy** ★ Top 5% • 1 year ago

I guess another option could be timed buckets with TTL. If a location update comes in at 4:07, we would add it to two sorted sets, "3:51" and "4:01" since it falls in both time buckets. Each of these sets would have a TTL of 30 seconds. For matching a ride request, we would fetch both sets and filter out drivers for which $\text{now} - \text{ts} > 10\text{s}$.

👍 0 [Reply](#)

D **denis** Premium • 1 month ago

Is the problem worth solving?

Let's say there are 10m drivers in the US and we store 1000 bytes per driver. That's 10Gb worth of data in RAM. A recent iPhone can fit that much.

We would have to scale out Redis to allow high rates of read/writes. Even if there is stale entry for every driver in every node it's still only 10Gb per node.

👍 1 [Reply](#)

 **Evan King** Admin • 1 month ago

This is the right question!

👍 0 [Reply](#)

C **CompatiblePeachPorpoise868** Premium • 1 year ago

IndirectSapphireMarmot112, as you have mentioned, Redis geospatial is a sorted set with geohashes. Thus, it's possible to use ZREM + GEOADD ($2 * \log N$) and update the driver's location whenever a new set of coordinates is being processed. Also, Redis libraries support pipelining, so both commands could be executed with a single roundtrip.

👍 0 [Reply](#)

F **FederalHarlequinLamprey277** • 1 year ago

+1!

👍 0 [Reply](#)

W **walnataraz** Premium • 6 months ago

From my side, I don't think using Redis For this use case is not proper. Since we need to manage

1. If driver is ASSIGNED or TAKING_THE_RIDE, then it should not exist in the `getNearestDriver` query.
2. We should have TTL to only consider newest location for driver, let's say within one minute window for `getNearestDriver`

I probably opt for In-Memory SQL database like VoltDB, DuckDB or Apache Ignite, SQLITE to make me more easy to maintain those driver states. Those database have their own Geospatial Index (but using RTREE). But if you concern about the Rtree balancing in high throughput uber cases, I probably just implement my geohash logic here by creating geohash column. Since this is in memory database, I would say it will have pretty much throughput with Redis

Surely redis possible to do it, but in interview, it would be hard to explain those details for case 1 and case 2

👍 0 [Reply](#)

A **AmazingTomatoBovid196** • 1 year ago

Hi Even,
Regarding:

"To solve the timeout issue, we can use a distributed lock implemented with an in-memory data store like Redis. "

I wanted to point out that Redis might not be the correct choice here, according to Martin Kleppmann:

<https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html>

👍 7 [Reply](#)



Ben Schmaltz • 11 months ago

I agree. Why move locking logic out of your db that was intended to be strongly consistent? We can already shard by location, so we don't have to make cross region locks.

You can optimize by finding available drivers that have been in the pending status past the timeout, so you don't have to rely on the cron job running. You'll have to add some complex error handling though.

Additionally, the driver rejecting or not responding, the client should send an API request most of the time.

👍 4 [Reply](#)



kstarikov ★ Top 1% • 7 months ago

Interestingly, Evan himself suggests this (much more viable) solution in Ticketmaster.

👍 0 [Reply](#)



Mike Choi ★ Top 5% • 1 year ago

Hey Evan, I think a possible correction is that PostGIS uses R-trees rather than quad trees, and quad trees are less optimized for uneven distributions as the trees are not able to automatically balance itself, whereas R-trees are optimized to self-balance to equalize depth of leaf nodes. Actually from my research, it doesn't seem like quad trees are used too often in location based services, but rather for gaming or imaging applications :)

👍 5 [Reply](#)



EconomicOrangeCicada244 Premium • 7 months ago

Why we are creating partialRide object for fare estimation. Fare estimation should not be persisted I believe as it needs to be scaled to multiple vehicle types in futures and there would rules of pricing that might be used. There should not be any write in db for fare estimation. Instead when user tries to book a ride, then create a ride object with status pending and send to ride matching service and when driver is assigned update the driver and ride db. So we would have 4 services, fare estimation, ride service, ride matching service and location service.

👍 5 [Reply](#)



bristleback Premium • 4 days ago

How would you ensure the fare is unchanged if you don't make any writes to DB?

Whenever you show a user "this ride is 5\$" user expects this specific fare for some reasonable period of time.

👍 0 [Reply](#)



WorthyTurquoiseTurtle357 ★ Top 10% • 1 year ago

How'd we handle case of send ride requests to the next driver if the initial one doesn't accept? I was thinking that if a driver doesn't accept a pick up request within the given time, a request could be sent to the ride matching service with the id of the request to that the ride matching service knows to find a new driver to match the user with. And taking the queue into account the priority on this request could be increased.

👍 4 [Reply](#)



WorthyTurquoiseTurtle357 ★ Top 10% • 1 year ago

Actually I just saw that it says "expiration allows the Ride Matching Service to consider the driver for new ride requests again." Is there a way to watch for or receive notifications from a distributed lock if the lock expires?

👍 2 [Reply](#)



Evan King Admin • 1 year ago

This is actually easier than that. It's basically a while loop on the ride matching server. While we still don't have a match, send the request to the next driver. Wait for a response for 10 seconds, then try the next one. If the service goes down, the ride request remains in the queue, and we resume from where we left off. The lock ensures that other services matching ride requests know not to send a request to a driver who already has an outstanding request.

👍 16 [Reply](#)

S

StrikingCyanLemming110 • 1 year ago

Hey Evan, thanks for making these great write-ups, very useful! I'm still a bit confused about the matching part. Isn't the notification from the APN an asynchronous request? What does wait for a response here mean? From what I'm understanding here, the matcher task runner pulls a task of the broker, find a match, locks a driver, sends an apn request, and does not commit offset until a match has been found. Where does the response come here?

👍 4 [Reply](#)



Evan King Admin • 7 months ago

The matcher doesn't wait for the APN response. When a driver accepts/declines via the app, they make a separate HTTP request to our backend. The matcher just needs to ensure the message was queued in APN (success/failure is immediate) before committing the offset. If APN fails, we can retry or move to the next driver. The actual driver response comes later through a different flow entirely.

👍 2 [Reply](#)

K

kushaq.2021 Premium • 5 months ago

@Evan King what I understand is that ride matching consumer just publishes request to all matched drivers asynchronously and commits offset post that commit message offset and moves to next one

So now we have x locked drivers whom we have sent request, when any driver accepts we check if ride was yet to be assigned if so assign/reject the drivers acceptance request, For this we should have pending rides as well in cache when any driver accepts it, we check if we have that pending ride in cache if yes update the ride (in db) and evict it from cache, in this way even if driver doesn't respond or respond later can be handled

Also prior to committing message service can check if it was accepted in between sending to all matched drivers, if yes good, otherwise put request in some retry table / priority queue which can act priority on it

👍 0 [Reply](#)

L

LogicalAmberBobolink693 • 1 year ago

My question is the same. @Evan can you please elaborate on this?

Although I don't think it's a good idea to not commit to the offset if the ride matching is not successful. We shouldn't let others wait to start their ride matching process if current one is taking more time.

👍 0 [Reply](#)

L

LovelyTomatoCephalopod342 Premium • 7 months ago

Agreed. Like finding a driver is a time consuming process (sending notification and waiting for their response).

👍 0 [Reply](#)

F

FederalHarlequinLamprey277 • 1 year ago

@SpotlessGoldAnteater112 A viable option could be:
while noMatch and there are potential drivers in line:

- driver = get next driver
- lock driver
- send notification to driver
- wait for driver's response(sleep for x seconds)
- check(pull) database to see if driver has responded. If not move on to the next driver.

👍 0 [Reply](#)

U

UnchangedCyanSwift203 • 1 year ago

How do we make sure that same driver doesn't get the same request if the ride matching server restarts ?

👍 3 [Reply](#)



Evan King Admin • 1 year ago

If the server restarts then we pull the ride match request back off the queue and go again. If the driver was locked, they'll be unlocked after the 10s TTL, at which case they may receive the request again, as is allowed.

👍 6 [Reply](#)



ASHISH AGGARWAL Premium • 7 months ago

@Evan King, Do you think using a workflow(like AWS lambda) could be beneficial here. Idea is that for each ride request, spawn a new workflow which will be responsible for fetching the list of drivers and sending the requests to them one by one.

👍 0 [Reply](#)



Kavish Dhamija • 1 year ago

Hi, would a better way be to use sqs as request queue and in matching service rather than waiting we can use visibility timeouts to do the same loop. In case of surge these timeouts can be updated

👍 1 [Reply](#)

F

FederalHarlequinLamprey277 • 1 year ago

+1!

SQS + visibility timeout, would allow scaling of consumers up/down depending on the length of the queue.

👍 1 [Reply](#)



WeiWangOneWay Premium • 4 months ago

Is a pub/sub topic for drivers matching more efficient than a while loop with wait? If the first 5 drivers all decline, the while loop run for more than one minute. With a message queue, after sending the ride request to the top driver, the process is over; and a decline PATCH request enqueues another matching request.

👍 0 [Reply](#)

R

ResidentOliveGamefowl522 • 1 year ago

From a resiliency standpoint we wouldn't have just a server, we'll have a fleet of servers with enough redundancy across the fleet (across geographic locations) to cater for these type of use cases in case a single instance goes down.

👍 0 [Reply](#)

N

Ninad Pradhan Premium • 7 months ago

Ive another solution, let me know your thoughts.

We dont use redis at all for ttl In this case, instead once an order is matched to a driver, we send a delayed another sqs message of 11 mins lets say if our typical timeout is 10 mins. Now once we get the sns message we simply query db

message or if it fails to say it our typical approach is to make it so that we get the msg message we simply query db with the order status; if its pending then find next driver in the queue.

Finally have some logic to say after 3 retries, its DLQ or something.

👍 0 [Reply](#)

P **PossibleBronzeDamselfly163** • 1 year ago

IMO, it could be an async state machine that decouples each stage of the ride initialization, matching, on ride, up for review etc

👍 0 [Reply](#)



FJ Sánchez • 1 year ago

That's exactly what I was thinking too, an async event-driven state machine that persists state using maybe DynamoDB or Redis.

👍 0 [Reply](#)



Evan King Admin • 1 year ago

true!

👍 0 [Reply](#)

[Show All Comments](#)

Schedule a mock interview

Meet with a FAANG senior+ engineer or manager and learn exactly what it takes to get the job.

[Schedule A Mock Interview](#)

Questions

[Meta SWE Interview Questions](#)
[Amazon SWE Interview Questions](#)
[Google SWE Interview Questions](#)
[OpenAI SWE Interview Questions](#)
[Engineering Manager \(EM\) Interview](#)

Links

[FAQ](#)
[Pricing](#)
[Gift Mock Interviews](#)
[Gift Premium](#)
[Become a Coach](#)

Legal

[Terms and Conditions](#)
[Privacy Policy](#)

Contact

[About Us](#)

Questions

Our Coaches

Product Support

Hello Interview Premium

7511 Greenwood Ave North

Unit #4238 Seattle

WA 98103

Learn

Learn System Design

Learn DSA

Learn Behavioral

Learn ML System Design

Learn Low Level Design

Guided Practice
